Avatar Generation Service — Final Architecture Plan

Kokokino Spoke App · MPFB2 + HiDream Pipeline

March 16, 2026 — Compiled from extended research session

Table of Contents

1. What We're Building
2. Why Now — The Market Opportunity
3. TalkingHead Technical Requirements
4. The Key Discovery: MPFB2 + met4citizen's Asset Packs
5. Architecture Decisions and Rationale
6. The Simplified MVP Pipeline
7. Three-Tier Deployment Architecture
8. Claude Code Development Phases
9. Prompt-to-Parameters: How User Input Becomes a Character
10. Storage Architecture
11. Complete Technology Stack with Licenses
12. Cost Analysis
13. Timeline
14. Risks and Mitigations
15. Post-MVP Roadmap

---

1. What We're Building

A Kokokino spoke app that generates unique, TalkingHead-ready 3D humanoid avatars from text prompts or images. Users describe a character in natural language or upload a reference image, and the service returns a complete GLB file with:

- A parametric humanoid body with unique proportions
- Separate mesh objects for eyes, teeth, tongue, eyelashes, eyebrows
- Mixamo-compatible skeletal rig
- 52 ARKit facial expression blend shapes

- 15 Oculus OVR viseme blend shapes for lip-sync
- Body and face textures derived from AI-generated concept art
- Compatible with TalkingHead, HeadTTS, and HeadAudio for real-time browser-based lip-sync

The service is essentially the successor to Ready Player Me, but instead of picking from a limited set of pre-made face/body options, every avatar is AI-influenced and parametrically unique.

---

2. Why Now — The Market Opportunity

Ready Player Me Is Gone

Netflix acquired Ready Player Me in December 2025 and shut down all public services on January 31, 2026. This included the Avatar Creator SDK, the PlayerZero SDK, and all API endpoints. Over 8,000 developers who integrated RPM into their projects lost their avatar pipeline overnight. VRChat users, game developers, metaverse platforms — all scrambling for alternatives.

The TalkingHead project itself noted this prominently: its README now carries a warning that RPM services will be wound down. The project historically relied on RPM for avatar creation, and the community now needs a new source of compatible avatars.

The Open Source Stack Just Reached Critical Mass

Five key releases in 2025 collectively enable what was previously impossible:

- **HiDream-I1** (April 2025): State-of-the-art open-source image generation, MIT licensed
- **MPFB2 stable release** (January 2025): Parametric human generator as a Blender plugin, with CC0 assets
- **MPFB2 viseme/faceunit packs** (February 2026): TalkingHead-compatible blend shapes by the TalkingHead author himself
- **UniRig** (SIGGRAPH 2025): Automatic rigging for any mesh (available as a future enhancement)
- **TRELLIS.2** (December 2025): State-of-the-art image-to-3D (available as a future enhancement)

Twelve months ago, building this service would have required hundreds of hours of manual 3D artist work per character. Today, the pipeline can be fully automated.

---

3. TalkingHead Technical Requirements

The TalkingHead JavaScript class (by met4citizen / Mika Suominen) requires avatars delivered as GLB files with the following specifications:

Mesh Requirements

- Full-body humanoid mesh
- Separate mesh objects for: head/body skin, left eye, right eye, upper teeth, lower teeth, tongue, eyelashes, eyebrows
- Clean topology suitable for blend shape deformation, particularly around eyes and mouth

Rig Requirements

- Mixamo-compatible bone hierarchy (bone names matching Mixamo convention)
- Properly painted skinning weights
- Supports FBX animations from Mixamo library

Blend Shape Requirements (Morph Targets)

52 ARKit facial expressions: eyeBlinkLeft, eyeBlinkRight, eyeLookDownLeft, eyeLookDownRight, eyeLookInLeft, eyeLookInRight, eyeLookOutLeft, eyeLookOutRight, eyeLookUpLeft, eyeLookUpRight, eyeSquintLeft, eyeSquintRight, eyeWideLeft, eyeWideRight, jawForward, jawLeft, jawRight, jawOpen, mouthClose, mouthFunnel, mouthPucker, mouthLeft, mouthRight, mouthSmileLeft, mouthSmileRight, mouthFrownLeft, mouthFrownRight, mouthDimpleLeft, mouthDimpleRight, mouthStretchLeft, mouthStretchRight, mouthRollLower, mouthRollUpper, mouthShrugLower, mouthShrugUpper, mouthPressLeft, mouthPressRight, mouthLowerDownLeft, mouthLowerDownRight, mouthUpperUpLeft, mouthUpperUpRight, browDownLeft, browDownRight, browInnerUp, browOuterUpLeft, browOuterUpRight, cheekPuff, cheekSquintLeft, cheekSquintRight, noseSneerLeft, noseSneerRight, tongueOut

15 Oculus OVR visemes: viseme_sil, viseme_PP, viseme_FF, viseme_TH, viseme_DD, viseme_kk, viseme_CH, viseme_SS, viseme_nn, viseme_RR, viseme_aa, viseme_E, viseme_I, viseme_O, viseme_U

Additional morph targets: mouthOpen, mouthSmile, eyesClosed, eyesLookUp, eyesLookDown

Animation Requirements

- Body animations as Mixamo-compatible FBX files
- Standard set: idle, walk, run, wave, nod, gesture, etc.

---

4. The Key Discovery: MPFB2 + met4citizen's Asset Packs

What MPFB2 Is

MPFB2 (MakeHuman Plugin For Blender) is a comprehensive parametric human character generator that runs as a Blender extension. It evolved from the MakeHuman standalone application into a fully featured Blender-native tool. It was released as stable (v2.0.8) in January 2025 and is actively maintained (v2.0.14 as of February 2026). It is compatible with Blender 4.2 through 5.1.

MPFB2 generates humanoid characters with:

- Hundreds of parametric body/face targets (sliders for every aspect of human morphology)
- Separate mesh objects for eyes, teeth, tongue, eyelashes, eyebrows (loaded from asset packs)
- Multiple rig options including a dedicated Mixamo rig
- Procedural skin and eye materials
- A comprehensive asset library (clothing, hair, accessories)
- An API-friendly service-oriented architecture with stateless singleton services

The met4citizen Connection

The critical discovery in our research is that **Mika Suominen (met4citizen)**, the creator of TalkingHead, HeadTTS, and HeadAudio, has personally contributed three asset packs to MPFB2 released in February 2026:

- **Faceunits 01**: ARKit-style facial expression shape keys
- **Visemes 01**: Microsoft-style visemes
- **Visemes 02**: Meta/Oculus-style visemes (the exact 15 visemes TalkingHead uses)

All released under CC0 (public domain).

This means the person who defined what TalkingHead needs has already created the exact assets that produce it from MPFB2 characters. The MPFB2 documentation even has a dedicated "Lip sync" page (planned for v2.0.15) describing how to load these shape keys and connect them to a lip sync addon.

The visemes02 pack uses the 15-shape Meta/ARKit naming convention (viseme_aa, viseme_CH, viseme_sil, etc.) — the exact names TalkingHead's code expects. This eliminates the need for any naming conversion, deformation transfer from ICT FaceKit, or manual blend shape creation.

Why This Changes Everything

In earlier iterations of this plan, the hardest unsolved problem was: "How do you automatically generate 67+ correctly-shaped blend shapes for any arbitrary humanoid mesh?" The proposed solutions involved deformation transfer algorithms, ICT FaceKit reference meshes, and significant quality risks.

With MPFB2 + met4citizen's asset packs, the problem is already solved. The blend shapes are pre-created for MPFB2's base mesh topology and parametrically adapt to any body/face configuration generated by MPFB2. They were created by the author of the target platform. There is no quality risk because the same person validated both sides.

Licensing

- MPFB2 source code: GPLv3 (compatible with Blender's license; runs inside Blender, never distributed separately)
- MPFB2 assets (all meshes, textures, targets, rigs): CC0 (public domain, no restrictions)
- Generated output (the characters): No restrictions. The MakeHuman team explicitly states that output from MPFB contains no trace of program logic, and since the assets are CC0, there is no limitation on what you can do with generated characters. You can sell them, modify them, use them commercially, and sublicense them.

This is not MIT, but the practical effect for a service that generates and sells avatars is equivalent. The GPLv3 only applies if you redistribute the MPFB2 plugin code itself, which you don't — it runs inside your Docker container, and users never receive it.

---

5. Architecture Decisions and Rationale

HiDream-I1 over FLUX (Image Generation)

Decision: Use HiDream-I1 as the primary concept image generator.

Why: HiDream-I1 is MIT licensed, which is critical for a commercial service. FLUX Kontext Dev uses a non-commercial license, and FLUX 2 Pro/Max are paid API-only. HiDream-I1 achieves state-of-the-art scores on HPS v2.1 (human preference), GenEval, and DPG benchmarks, outperforming FLUX Dev. Its four-encoder architecture (OpenCLIP ViT-bigG, OpenAI CLIP ViT-L, T5-v1.1-xxl, Llama-3.1-8B-Instruct) allows routing structured character attributes to different encoders for finer control — tags to CLIP, descriptions to Llama. The FP8 variant fits on an RTX 4090.

HiDream-E1.1 (MIT licensed) provides instruction-based image editing for refinement ("make the hair longer," "add a scar"). It's not as precise as FLUX Kontext for consistency, but it's free, open, and sufficient for single-image edits.

FLUX Kontext remains available as an optional future enhancement for multi-view turnaround sheets if needed for the TRELLIS.2 path post-MVP.

MPFB2 over Custom Templates (Character Generation)

Decision: Use MPFB2 as the character generation engine rather than building custom templates from scratch.

Why: Creating production-quality humanoid templates from scratch — with proper face topology, edge loops, separate face components, clean UVs, Mixamo rig, and skinning weights — is 40–80 hours of skilled 3D artist work per template. MPFB2 provides all of this through a decade-refined parametric system with hundreds of variation targets, and the output is guaranteed to have correct topology because it's been validated across thousands of users.

More importantly, MPFB2 has the met4citizen asset packs for TalkingHead-compatible blend shapes. No custom template would have these without re-solving the blend shape generation problem from scratch.

MPFB2's service-oriented architecture (HumanService, RigService, AssetService as importable Python singletons) is designed to be called programmatically, making headless Blender integration feasible.

TRELLIS.2 as Post-MVP Only (3D Mesh Generation)

Decision: TRELLIS.2 is not in the MVP pipeline. It's reserved for future features.

Why: MPFB2 eliminated the need for AI mesh generation for humanoid characters. MPFB2 generates humanoids from parameters rather than from images, and the output is superior for our use case (guaranteed topology, pre-fitted components, built-in rigging). TRELLIS.2 would be needed for:

- Rigid accessories/props that aren't in MPFB2's library (helmets, weapons, unique items)
- Non-humanoid bodies that exceed MPFB2's parametric range
- Higher-quality texture generation than projection-based approaches

None of these are MVP requirements. Removing TRELLIS.2 from the MVP eliminates an RTX 5090 endpoint (~$0.89/hr), reduces Docker complexity from 5 containers to 2, and removes the riskiest pipeline stages (retopology, face separation, deformation transfer).

RunPod Serverless over DigitalOcean (GPU Compute)

Decision: Use RunPod Serverless for all GPU and heavy compute.

Why: RunPod Serverless scales to zero — when nobody is generating avatars, GPU cost is $0. DigitalOcean GPU Droplets are always-on VMs that charge whether active or idle. For a service

with bursty demand (someone generates an avatar, then nothing for a while), serverless is dramatically more cost-effective.

RunPod offers RTX 4090 at ~$0.69/hr, charges per-second, has 30+ global data centers, supports custom Docker containers, provides network storage shared across endpoints, and has pre-built templates for AI workloads. The serverless model also enables automatic scaling if demand grows — more workers spin up during peaks without any infrastructure management.

Podman over Docker (Container Runtime)

Decision: Use Podman for building and managing containers.

Why: Docker Desktop requires paid subscriptions for companies with 250+ employees. While this doesn't apply to a solo developer today, the licensing direction signals commercial tightening. Podman is Apache 2.0 licensed, fully free, and a drop-in replacement — you can alias `docker` to `podman` and all commands work identically. It reads Dockerfiles natively, pushes to the same OCI-compliant registries, and produces identical container images.

Podman is also technically superior: daemonless (no background service), rootless by default (more secure), and lighter on system resources. It's backed by Red Hat with long-term commitment.

The container images produced by Podman are OCI-standard — RunPod, Quay.io, and every other container platform consumes them identically to Docker images.

Quay.io over GHCR or Docker Hub (Container Registry)

Decision: Use Quay.io for hosting container images.

Why: Quay.io is Red Hat's container registry, free for public repositories with no bandwidth or storage limits. It pairs naturally with Podman (both Red Hat, both Apache 2.0).

GHCR (GitHub Container Registry) was the initial recommendation but has a "currently free" status that could change — GitHub (Microsoft) hasn't committed to permanent free container hosting and reserves the right to start charging with 30 days notice. Docker Hub has pull rate limits (100 per 6 hours on free tier) that would be problematic for RunPod cold starts.

Quay.io has a stated free-for-public-repos policy, no pull rate limits, and includes vulnerability scanning. It's the consistent choice for an open-source-first stack.

Backblaze B2 over CloudFlare R2 (Asset/Avatar Storage)

Decision: Use Backblaze B2 for storing generated avatars and pipeline artifacts.

Why: The original plan suggested CloudFlare R2 for its zero-egress pricing and built-in CDN. But examining the actual usage pattern — users download their GLB file once or twice, then use it in their own projects — the CDN advantage is irrelevant. Users aren't streaming avatars from our servers on every page load.

B2 is 2.5x cheaper for storage ($0.006/GB/month vs R2's $0.015/GB). This matters because we're keeping intermediate pipeline artifacts (concept images, variant rejects, parameter files, pre-texture meshes, final GLBs) to enable future re-entry into the pipeline at any stage. A single avatar's full artifact set is ~50–100 MB.

B2 provides 1GB free egress per day (~30GB/month), which is ample for download-once patterns. Its S3-compatible API means the same Node.js SDK works for both B2 and R2 — easy to switch later if CDN becomes needed.

If CDN is ever required (e.g., a public gallery where browsers preview many avatars), CloudFlare's free CDN tier can be placed in front of B2 via the Bandwidth Alliance (free traffic between B2 and CloudFlare).

Babylon.js over Three.js (3D Viewer)

Decision: Use Babylon.js for the browser-based 3D preview and avatar interaction.

Why: The user's preference, fitting the Kokokino ecosystem. Babylon.js has full GLB morph target support — the GLTF 2.0 loader reads morph target names and creates MorphTarget objects with programmable influence values (0.0–1.0). Skeleton animation from GLB works out of the box.

For the TalkingHead lip-sync demo, two approaches are viable:

1. Native Babylon.js implementation: HeadAudio/HeadTTS output viseme weights → set `morphTarget.influence` in Babylon.js animation loop
2. TalkingHead iframe embed for rapid MVP (TalkingHead itself uses Three.js internally)

Approach 1 is recommended for production; approach 2 for initial testing.

Kokokino Spoke App Skeleton (App Framework)

Decision: Build on the kokokino/spoke_app_skeleton template.

Why: The skeleton provides SSO integration with Kokokino Hub, subscription management middleware, MongoDB collections, Mithril.js reactive components, and deployment patterns for Meteor Galaxy. This eliminates weeks of boilerplate for authentication, billing, and session management. The avatar generation features layer on top of the existing structure.

Blender 5.0 in Cloud, 3.6 Locally (Mesh Processing)

Decision: Run Blender 5.0 in RunPod Docker containers for production. Use Blender 3.6 on the 2013 Mac Pro for development script prototyping.

Why: The 2013 Mac Pro can only run up to macOS Monterey (12.x), which limits Blender to 3.6 (released June 2023, support ended June 2025). Blender 4.x and 5.x require macOS 13.0+. However, the bpy Python API for core mesh operations (shape keys, armatures, GLTF export,

object manipulation) is stable across 3.6 through 5.0. Scripts developed on 3.6 work on 5.0 with minimal changes, as long as they avoid 5.0-only features (ACES color management, new geometry nodes, SDF nodes — none of which the pipeline uses).

MPFB2 requires Blender 4.2+, so it cannot run on the Mac Pro locally. All MPFB2 testing and production runs on Blender 5.0 in RunPod containers. Non-MPFB2 Blender scripts (texture baking, GLB export tweaks, mesh analysis) can be prototyped on 3.6 locally and validated on 5.0 via RunPod.

Blender 5.0 is the current stable release (November 18, 2025). Blender 5.1 entered Release Candidate on March 17, 2026. Docker containers will pin to 5.0 initially.

---

6. The Simplified MVP Pipeline

The pipeline has been progressively simplified through research. The final MVP involves only two cloud endpoints.

```
USER INPUT
  |
  |  "A cyberpunk female warrior with blue hair and facial scars"
  |  (or: uploads a reference image)
  |
  ▼
METEOR GALAXY (Kokokino Spoke App)
  |
  |  Step 1: PROMPT DECOMPOSITION (runs on Galaxy, no GPU)
  |  Claude API or rule-based parser extracts:
  |  → body type, gender, age, build
  |  → face shape, features, skin tone
  |  → hair style and color
  |  → clothing selection
  |  → special features (scars, tattoos, etc.)
  |  Maps to MPFB2 target values + HiDream prompt
  |
  ├──────────────────────────────────────────────────────────┐
  |  Step 2: CONCEPT IMAGE (RunPod: HiDream-I1 endpoint)    |
  |  Generate full-body T-pose character, white background   |
  |  Generate face close-up for texture detail               |
  |  ~10 seconds per image on RTX 4090                       |
  |  User sees preview → approve / regenerate / edit         |
  └──────────────────────────────────────────────────────────┘
  |
  |  (User approves)
  |
  ├──────────────────────────────────────────────────────────┐
  |  Step 3: CHARACTER GENERATION                            |
  |  (RunPod: Blender 5.0 + MPFB2 endpoint, CPU only)       |
  |                                                          |
  |  a. Create base human via MPFB2 HumanService             |
  |     Apply body targets from prompt decomposition         |
  |     Apply face targets from prompt decomposition         |
  |                                                          |
  |  b. Add system assets via MPFB2 AssetService             |
  |     Eyes, teeth, tongue, eyelashes, eyebrows             |
  |     Select skin material                                 |
  |                                                          |
  |  c. Add Mixamo rig via MPFB2 RigService                  |
  |     Mixamo-compatible bone hierarchy                     |
  |     Automatic weight painting                            |
  |                                                          |
```

```
  |  d. Load face shape keys                                  |
  |     Faceunits 01 → ARKit expression blend shapes          |
  |     Visemes 02 → Oculus lip-sync blend shapes             |
  |     Additional targets (mouthOpen, eyesClosed, etc.)      |
  |                                                           |
  |  e. Select clothing from MPFB2 library                    |
  |     Based on prompt decomposition output                  |
  |                                                           |
  |  f. Select hair from MPFB2 library                        |
  |     Based on prompt decomposition output                  |
  |                                                           |
  |  g. Project textures from concept image                   |
  |     Face close-up → face UV island                        |
  |     Body concept → body UV islands                        |
  |     Bake to texture maps                                  |
  |                                                           |
  |  h. Export as GLB                                         |
  |     All morph targets with correct naming                 |
  |     Skeleton with Mixamo bone names                       |
  |     PBR materials and textures                            |
  |     ~60-90 seconds total on CPU                           |
  └───────────────────────────────────────────────────────────┘

  |
  |  Step 4: UPLOAD + DELIVER
  |  Upload GLB + concept images to Backblaze B2
  |  Store metadata + B2 URLs in MongoDB
  |  Notify user via DDP reactive publication
  |  User previews in Babylon.js viewer
  |  User downloads GLB file
  |
  ▼
USER'S PROJECT
  TalkingHead / HeadTTS / HeadAudio
  Game engine (Unity, Unreal, Godot)
  VRChat, social VR platforms
```

Total generation time: ~70-100 seconds (10s image gen + 60-90s character gen) **Total compute cost per avatar**: ~$0.01-0.02 **Number of RunPod endpoints**: 2 (HiDream + Blender/MPFB2) **Number of Docker containers to maintain**: 2

---

7. Three-Tier Deployment Architecture

Tier 1: Development Machine (2013 Mac Pro)

What it does:

- Code editing with Claude Code
- Meteor app development and local testing
- Babylon.js frontend development in browser
- Blender 3.6 for non-MPFB2 script prototyping
- Python development (prompt decomposition, API logic, RunPod client)
- Podman for building container images
- Git for version control
- SSH into RunPod GPU Pods for interactive GPU testing

What it cannot do:

- Run Blender 4.x or 5.x (macOS version too old)
- Run MPFB2 (requires Blender 4.2+)
- Run any ML models (no CUDA GPU)

Tier 2: Meteor Galaxy (Web Application)

What it hosts:

- Kokokino spoke app (Meteor + Mithril.js)
- Babylon.js 3D avatar preview and interaction
- REST/DDP API for avatar generation requests
- Job queue and real-time status tracking (MongoDB)
- User accounts via Kokokino Hub SSO
- Subscription tier enforcement
- RunPod API orchestration (calls endpoints in sequence)
- Avatar gallery and user history
- Serves static app assets

What it does NOT host:

- GPU compute (delegated to RunPod)
- Binary file storage (delegated to Backblaze B2)

- Container images (hosted on Quay.io)

Cost: $0 (sandbox) to $30/month (professional plan)

Tier 3: RunPod Serverless (GPU Compute)

Endpoint 1: HiDream-I1

- Container: Podman-built, pushed to Quay.io
- Base: pytorch/pytorch with CUDA 12.4
- Models: HiDream-I1-Dev (or Full Q8) baked into image
- GPU: RTX 4090 (24GB VRAM)
- Cost: ~$0.69/hr, ~$0.002 per image (~10 seconds)
- Scales to zero when idle

Endpoint 2: Blender 5.0 + MPFB2

- Container: Podman-built, pushed to Quay.io
- Base: Ubuntu 24.04 + Blender 5.0 headless
- Extensions: MPFB2 pre-installed
- Asset packs: All CC0 packs pre-loaded (~2–3 GB)
- Pipeline scripts: Character generation, texture projection, GLB export
- GPU: CPU only (~$0.20/hr) — no GPU needed for mesh operations
- Cost: ~$0.003 per avatar (~60–90 seconds)
- Scales to zero when idle

Shared: RunPod Network Storage for pipeline artifacts in transit between endpoints

Communication Flow

```
Mac Pro → (git push) → GitHub
Mac Pro → (podman push) → Quay.io
Mac Pro → (meteor deploy) → Galaxy
Mac Pro → (SSH) → RunPod GPU Pods (development only)

Galaxy → (HTTP API) → RunPod Endpoints (job requests)
RunPod → (webhook) → Galaxy (job completion)
RunPod → (S3 API) → Backblaze B2 (upload artifacts)
Galaxy → (S3 API) → Backblaze B2 (generate download URLs)

User Browser → (DDP/WebSocket) → Galaxy (real-time updates)
```

```
User Browser → (HTTPS) → Backblaze B2 (download GLB files)
User Browser → (HTTPS) → Galaxy (Babylon.js app, API calls)
```

---

8. Claude Code Development Phases

Phase 0: Foundation & MPFB2 Validation (1-2 weeks)

This is the single most important phase. It answers: "Can MPFB2 generate a TalkingHead-compatible avatar headlessly?"

On Mac Pro (local):

```
SPOKE APP SCAFFOLD:
- Fork kokokino/spoke_app_skeleton
- npm install @babylonjs/core @babylonjs/loaders
- Create AvatarGenerator Mithril component (text input + image upload)
- Create AvatarPreview Mithril component (Babylon.js GLB viewer)
  Test with a known-good RPM-compatible GLB file
  Verify morph targets load and can be driven via influence values
- Create JobsCollection + DDP publications for real-time progress
- Write RunPod API client module (HTTP calls)
- Write prompt decomposition module (Claude API or rule-based)
- Test full Meteor app runs locally on Mac Pro

BLENDER SCRIPT PROTOTYPING (Blender 3.6):
- Write basic bpy scripts for:
  - Loading a mesh, listing shape keys, exporting GLB
  - Reading a JSON params file and applying values
  - These test the non-MPFB2 parts of the pipeline
```

On RunPod GPU Pod (~$5-10 spend, SSH from Mac):

```
MPFB2 HEADLESS VALIDATION (critical experiment):
- Spin up a CPU or cheap GPU Pod
- Install Blender 5.0
- Install MPFB2 extension
- Download and install asset packs:
  - makehuman_system_assets (267 MB) — REQUIRED
  - faceunits01 (0.2 MB) — REQUIRED
  - visemes02 (0.1 MB) — REQUIRED
  - visemes01 (0.1 MB) — useful
  - skins01 + skins02 (skin textures)
  - hair01 (basic hair)
- Write headless test script:
  1. Import MPFB2 services:
     from mpfb.services.humanservice import HumanService
  2. Call HumanService.create_human() in --background mode
  3. If it crashes → identify the bpy.ops call that needs context override
     → patch or wrap with context override
  4. Apply body/face targets via shape key values
  5. Load system assets (eyes, teeth, tongue, eyelashes)
  6. Add Mixamo rig
  7. Load faceunits01 shape keys (ARKit)
  8. Load visemes02 shape keys (Oculus)
  9. Export as GLB with morph targets
  10. Download GLB to Mac Pro
  11. Load in Babylon.js viewer → verify morph targets present
  12. Load in TalkingHead test page → verify lip-sync works

HIDREAM-I1 SMOKE TEST:
- On same or separate Pod with RTX 4090
- Install HiDream-I1-Dev via diffusers
- Generate test character concept images
- Measure VRAM usage and generation time
- Verify image quality is suitable for texture projection

RESULT: If steps 1-12 above work → the core architecture is proven.
If step 2 crashes → assess how much patching is needed.
```

Deliverable: Working spoke app skeleton with Babylon.js viewer. Proven or disproven MPFB2 headless pipeline. HiDream-I1 benchmarks.

Phase 1: MPFB2 Pipeline Automation (2–3 weeks)

Goal: Complete character generation pipeline callable via RunPod API.

On Mac Pro:

```
PIPELINE ORCHESTRATOR (Meteor server code):
- server/avatar-pipeline.js:
  Orchestrate multi-step generation via RunPod API calls
  Step 1: Call HiDream endpoint → get concept images
  Step 2: Update job status, send preview to user via DDP
  Step 3: Wait for user approval
  Step 4: Call Blender/MPFB2 endpoint → get GLB
  Step 5: Upload GLB to Backblaze B2
  Step 6: Update MongoDB with B2 URLs
  Step 7: Notify user via DDP

PROMPT DECOMPOSITION:
- imports/lib/prompt-decomposer.js:
  Parse "cyberpunk female warrior with blue hair" into:
  {
    gender: 0.1,           // MPFB2 macro target (0=female, 1=male)
    age: 0.4,              // young adult
    weight: 0.3,           // lean/athletic
    muscle: 0.7,           // muscular
    proportions: 0.5,      // average
    face_targets: {
      jaw_width: 0.6,      // angular jaw
      cheekbone_prominence: 0.7,
      eye_size: 0.5,
      nose_width: 0.4,
      lip_fullness: 0.5
    },
    hair: "short_asymmetric",  // maps to MPFB2 hair asset
    skin: "medium_female",      // maps to MPFB2 skin
    clothing: "sci_fi_suit",    // maps to MPFB2 clothing asset
    hair_color: [0.0, 0.3, 1.0],  // RGB for blue
    hidream_prompt: "full body T-pose female warrior, blue asymmetric hair,
      facial scars on left cheek, dark armored bodysuit with neon blue accents,
      white background, studio lighting, game character concept art"
  }

BABYLON.JS INTEGRATION:
- imports/lib/babylon-loader.js:
  Load GLB, enumerate morph targets, provide control API
  Slider UI for testing each blend shape
  Basic animation playback for skeleton animations
```

On RunPod (build and deploy containers):

```
BLENDER/MPFB2 CONTAINER:
- Write Dockerfile (using Podman):
  FROM ubuntu:24.04
  Install Blender 5.0 headless
  Install MPFB2 extension
  Copy all CC0 asset packs (~2-3 GB)
  Copy pipeline scripts
  Install RunPod Python SDK
  Copy handler.py

- handler.py accepts tasks:
  "create_character": params → MPFB2 → GLB
  "apply_texture": images + mesh → textured GLB
  "export_glb": final export with all morph targets

- Build: podman build -t quay.io/youruser/mpfb2-worker:v1 .
- Push: podman push quay.io/youruser/mpfb2-worker:v1
- Deploy as RunPod Serverless endpoint (CPU)

HIDREAM CONTAINER:
- Similar Dockerfile with HiDream-I1-Dev
- handler.py: prompt → image(s)
- Deploy as RunPod Serverless endpoint (RTX 4090)
```

Deliverable: End-to-end pipeline: text prompt → concept image → approval → character generation → GLB download.

Phase 2: Texturing & Polish (2–3 weeks)

Goal: AI-generated textures on MPFB2 characters. User-facing polish.

```
TEXTURE PROJECTION:
- Blender script: project HiDream concept image onto MPFB2 character UVs
- Face close-up → face UV island (highest detail)
- Body concept → body UV islands
- Bake to 2048x2048 texture maps
- Handle UV seam blending

ITERATIVE REFINEMENT:
- Add HiDream-E1.1 endpoint for "make hair longer" type edits
- Approval flow: user sees concept → approves or requests changes
- Regeneration with modified prompt

USER INTERFACE POLISH:
- Style selector: realistic / cartoon / anime
- Body customization sliders (exposed MPFB2 targets)
- Hair/clothing selection from MPFB2 library
- Avatar gallery (list user's generated avatars)
- Download options (GLB, separate textures)
- TalkingHead demo embed (iframe or native Babylon.js lip-sync)

SUBSCRIPTION INTEGRATION:
- Gate generation behind Kokokino subscription tiers:
  Free: 3 avatars/month
  Pro: 50 avatars/month
  Studio: unlimited + API access
```

Phase 3: Production Deploy (1-2 weeks)

```
DEPLOYMENT:
- meteor deploy to Galaxy (production domain)
- Configure Kokokino Hub SSO for production
- Set up Backblaze B2 bucket with appropriate CORS
- Set up RunPod Serverless endpoints (production)
- Monitoring: RunPod dashboard + Galaxy APM
- Error handling: graceful failures with user notification

TESTING:
- Generate 50+ avatars across different prompts/styles
- Validate each in TalkingHead test page
- Validate in Babylon.js viewer
- Load test: simulate concurrent generation requests
- Verify Mixamo animations retarget correctly
```

9. Prompt-to-Parameters: How User Input Becomes a Character

This is the "intelligence" layer of the service. The prompt decomposer takes natural language (or an image) and produces two outputs: MPFB2 target values for the 3D character, and a HiDream prompt for the concept image.

Text Input Path

User types: "A stocky middle-aged man with a wide jaw and small eyes, wearing a leather jacket"

The LLM (Claude API) parses this into structured JSON:

```json
{
  "mpfb_targets": {
    "macrodetails/Gender": 1.0,
    "macrodetails/Age": 0.6,
    "macrodetails/Weight": 0.7,
    "macrodetails/Muscle": 0.5,
    "macrodetails/Proportions": 0.4,
    "head/head-square": 0.6,
    "jaw/jaw-width-max": 0.7,
    "eyes/eyes-size-small": 0.6,
    "forehead/forehead-height-min": 0.3
  },
  "assets": {
    "skin": "skins02/middleage_caucasian",
    "eyes": "brown",
    "eyebrows": "eyebrow003",
    "eyelashes": "eyelashes01",
    "hair": "hair01/short_receding",
    "clothing": "suits01/leather_jacket"
  },
  "hidream_prompt": "full body T-pose stocky middle-aged man, wide square jaw, small
}
```

The MPFB2 target names map to actual shape key names on the basemesh. The asset names map to files in the asset pack directories. This mapping is the core intellectual work of the prompt decomposer — building a dictionary between natural language descriptors and MPFB2's target/asset system.

Image Input Path

User uploads a photo or reference image. The pipeline:

1. Generates a HiDream-I1 concept image in the character's style (using IP-Adapter or img2img from the reference)
2. Uses MediaPipe FaceMesh on the reference image to detect facial proportions
3. Maps detected proportions to MPFB2 face targets:
   - Wider-than-average eye spacing → increase eye_spacing target
   - Prominent cheekbones → increase cheekbone target
   - Round face → increase face_roundness target
4. Body proportions estimated from full-body reference (height/width ratio, shoulder width, etc.)

This image-to-parameters mapping is a Phase 2/3 feature. MVP focuses on text input.

---

10. Storage Architecture

Backblaze B2 Bucket Structure

```
b2://avatar-service/
  avatars/
    {userId}/
      {avatarId}/
        hero.png              # Approved HiDream concept (200KB-1MB)
        face_closeup.png      # High-res face for texture (500KB-2MB)
        variants/             # Rejected concept alternatives
          variant_01.png
          variant_02.png
        params.json           # MPFB2 targets + prompt + settings
        avatar.glb            # Final TalkingHead-ready GLB (5-20MB)
        avatar_lowpoly.glb    # Low-poly variant if generated
        thumbnail.jpg         # 256x256 gallery thumbnail (20-50KB)
        metadata.json         # Timestamps, pipeline state, quality scores
```

Why keep intermediate files: The params.json stores exact MPFB2 slider values. If a user wants to "remix" their character later (same body, different face; same face, different hair), we reload those params and re-enter the pipeline at the right stage. This enables a premium "character editing" feature without re-running the full pipeline.

Pipeline state tracking enables re-entry:

```json
{
  "conceptGenerated": true,
  "conceptApproved": true,
  "meshGenerated": true,
  "textured": true,
  "blendshapesApplied": true,
  "exported": true
}
```

MongoDB on Galaxy (Metadata Only)

```javascript
```

```
AvatarsCollection = {
  _id: "abc123",
  userId: "user456",
  prompt: "A stocky middle-aged man...",
  style: "realistic",
  params: { /* MPFB2 target values */ },
  status: "complete",
  files: {
    hero: "https://f003.backblazeb2.com/.../hero.png",
    glb: "https://f003.backblazeb2.com/.../avatar.glb",
    thumbnail: "https://f003.backblazeb2.com/.../thumbnail.jpg"
  },
  pipelineState: { /* re-entry state */ },
  qualityScore: 0.92,
  createdAt: ISODate("2026-04-15T..."),
  generationTimeMs: 85000
}

JobsCollection = {
  _id: "job789",
  userId: "user456",
  avatarId: "abc123",
  status: "generating_mesh",  // reactive, drives UI progress
  progress: 55,               // 0-100
  currentStep: "Adding Mixamo rig",
  runpodJobIds: {
    hidream: "rp_xxx",
    blender: "rp_yyy"
  },
  createdAt: ISODate("..."),
  updatedAt: ISODate("...")
}
```

Cost Estimate

At 1,000 users with 5 avatars each, ~50-100MB per avatar artifact set:

- Storage: 250-500 GB × $0.006/GB = $1.50-3.00/month
- Egress: ~30 GB/month (within free 1GB/day tier) = $0
- Total B2 cost: ~$2-3/month

---

11. Complete Technology Stack with Licenses

Core Pipeline (All commercially permissive)

Component	Version	License	Role	Cost
Meteor	3.x	MIT	App framework (spoke app)	Free
Mithril.js	2.x	MIT	UI components	Free
Babylon.js	7.x	Apache 2.0	3D viewer + morph target animation	Free
HiDream-I1	Dev/Full	MIT	Concept image generation	Free (self-hosted)
HiDream-E1.1	Full	MIT	Image editing/refinement	Free (self-hosted)
MPFB2	2.0.14+	GPLv3 (code), CC0 (assets+output)	Parametric character generation	Free
Blender	5.0	GPL	Headless mesh processing engine	Free
Podman	Latest	Apache 2.0	Container building and management	Free
RunPod SDK	Latest	MIT	Serverless GPU orchestration	Free (pay for compute)

Asset Packs (All CC0 — public domain)

Pack	Author	Size	Contents
MakeHuman system assets	MakeHuman team	267 MB	Eyes, teeth, tongue, eyelashes, skins, proxies, basic hair/clothes
Faceunits 01	Mika Suominen (met4citizen)	0.2 MB	ARKit facial expression targets
Visemes 01	Mika Suominen (met4citizen)	0.1 MB	Microsoft viseme targets

Pack	Author	Size	Contents
Visemes 02	Mika Suominen (met4citizen)	0.1 MB	Oculus/Meta viseme targets
Skins 01-03	Various CC0 authors	~100 MB	Male, female, non-natural skin textures
Hair 01-03	Various CC0/CC-BY authors	~200 MB	Low-poly, high-poly, misc hair
Clothing packs	Various CC0/CC-BY authors	~1 GB	Shirts, pants, suits, dresses, etc.
Body deformations	Various CC0 authors	~50 MB	Arms, hands, nose, ears, cheeks

Infrastructure Services

Service	Role	Cost
Meteor Galaxy	Web app hosting	$0-30/month
RunPod Serverless	GPU compute (HiDream) + CPU compute (Blender)	~$0.01-0.02/avatar, $0 idle
Quay.io	Container image registry	$0 (public repos)
Backblaze B2	Avatar and artifact storage	~$2-5/month at moderate scale
Kokokino Hub	SSO + billing integration	Part of Kokokino ecosystem
GitHub	Source code repository	$0 (free tier)
MongoDB Atlas or Galaxy Managed	Database	$0-8/month

Post-MVP (available when needed, not in initial pipeline)

Component	License	Role
TRELLIS.2	MIT	Rigid accessory/prop generation from images

Component	License	Role
UniRig	MIT	Auto-rigging for non-MPFB2 meshes
FLUX Kontext	Non-commercial (Dev) / Paid (Pro)	Multi-view turnaround for enhanced 3D
Instant Meshes	BSD	Auto-retopology for TRELLIS.2 output
MediaPipe	Apache 2.0	Face landmark detection (image-to-params)
rembg	MIT	Background removal

12. Cost Analysis

Development Phase

Item	Cost
Mac Pro (existing)	$0
Galaxy sandbox plan	$0
RunPod Pods for MPFB2 validation (~8 hours)	~$5-10
RunPod Serverless testing (50 test avatars)	~$1-5
Quay.io (public repos)	$0
Backblaze B2 (development)	$0-1
Total development cost	**~$6-16**

Production — Per Avatar Cost Breakdown

Step	Compute	Duration	Cost
Prompt decomposition	Galaxy CPU	~1 second	~$0
HiDream hero image	RTX 4090 @ $0.69/hr	~10 seconds	$0.002
HiDream face close-up	RTX 4090 @ $0.69/hr	~10 seconds	$0.002
MPFB2 character gen	CPU @ $0.20/hr	~30 seconds	$0.002

Step	Compute	Duration	Cost
Texture projection	CPU @ $0.20/hr	~30 seconds	$0.002
GLB export	CPU @ $0.20/hr	~15 seconds	$0.001
B2 upload	Network	~5 seconds	$0
Total per avatar		**~100 seconds**	**~$0.009**

Under one cent per avatar. Even with RunPod overhead (cold start time, minimum billing), realistic cost is $0.02-0.05 per avatar.

Production — Monthly Cost by Scale

Scale	Galaxy	RunPod	B2 Storage	Total
Demo (5/day)	$0	~$3	$1	~$4/mo
Small (50/day)	$15	~$30	$3	~$48/mo
Medium (500/day)	$30	~$300	$10	~$340/mo
Large (5000/day)	$100	~$3000	$50	~$3150/mo

Revenue Model (Kokokino Subscription)

At ~$0.02-0.05 per avatar, even a $5/month subscription that includes 20 avatars has massive margins. The infrastructure cost per avatar is negligible; the value is in the convenience and quality of the automated pipeline.

---

13. Timeline

Phase	Duration	What	Key Milestone
Phase 0	1–2 weeks	Foundation + MPFB2 validation	"Can MPFB2 generate a TalkingHead GLB headlessly?"
Phase 1	2–3 weeks	Pipeline automation + containers	End-to-end: prompt → GLB via RunPod
Phase 2	2–3 weeks	Texturing + UI polish + subscriptions	Users can generate and download avatars

Phase	Duration	What	Key Milestone
Phase 3	1–2 weeks	Production deploy + testing	Live service on Galaxy
Total to launch	**~6–10 weeks**		

The Phase 0 validation experiment is the critical gate. If MPFB2 works headlessly (which the architecture strongly suggests), the rest is engineering execution with known tools. If it doesn't work headlessly, we'd need to assess how much patching is required — but this would be known within the first day of Phase 0.

---

14. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
MPFB2 fails to run headlessly	Low-Medium	Critical	Phase 0 validates this first. If bpy.ops calls fail, wrap with context overrides. Worst case: patch specific MPFB2 service functions (code is open source).
Faceunits 01 doesn't contain all 52 ARKit shapes	Low	High	Verify in Phase 0. If subset, supplement with deformation transfer for missing shapes. Met4citizen's involvement makes full coverage likely.
Viseme naming doesn't match TalkingHead exactly	Very Low	Medium	Verify in Phase 0. Same author created both — naming match is near-certain. If different, simple rename script in export.
MPFB2 Mixamo rig weight painting quality	Medium	Medium	Known to be "crude in places" per release notes. Works for common scenarios. For extreme body types, may need weight cleanup scripts.
HiDream-I1 concept images poor for texture projection	Medium	Medium	Can improve with prompt engineering, ControlNet, or fallback to MPFB2's procedural skin materials. Concept images are for appearance variation, not geometry.
RunPod cold start latency	Low	Low	FlashBoot reduces to sub-200ms for pre-warmed workers. First request after long idle may take 30-60s for model loading.

Risk	Likelihood	Impact	Mitigation
Blender 3.6/5.0 script incompatibility	Low	Low	Write to common bpy API subset. Test on 5.0 via RunPod before shipping. Core mesh operations are stable across versions.
Galaxy WebSocket limits for real-time progress	Low	Low	Galaxy supports DDP/WebSocket natively. Alternatively, poll job status via Meteor methods.

---

15. Post-MVP Roadmap

Near-term (Months 2-3)

- HiDream-E1.1 endpoint for iterative character refinement
- Image-to-parameters: upload a photo → extract face proportions → generate matching avatar
- More hair and clothing variety from MPFB2 asset packs
- Animation library: attach Mixamo animations to generated avatars
- Native Babylon.js lip-sync (replace TalkingHead iframe)

Medium-term (Months 4-6)

- TRELLIS.2 endpoint for rigid accessories and props
- Custom texture painting: AI-enhanced skin details (scars, tattoos, makeup)
- Body deformation packs for more extreme body types
- Character editing: reload params.json, modify specific aspects, re-export
- Public avatar gallery with community sharing

Long-term (Months 6+)

- Non-humanoid characters (TRELLIS.2 + UniRig path for novel body types)
- Custom clothing generation (TRELLIS.2 → shrinkwrap fitting)
- Video-to-avatar: extract character from video reference
- Multi-character scene generation
- API access for developer integration (successor to RPM SDK)
- Fine-tune HiDream-I1 on character art dataset

- Fine-tune MPFB2 target mapping for better prompt adherence

Appendix A: Docker Container Specifications

Container 1: HiDream-I1 Worker

```dockerfile
FROM pytorch/pytorch:2.4.0-cuda12.4-cudnn9-runtime

RUN pip install diffusers transformers accelerate runpod safetensors

# Download model weights at build time
RUN python -c "
from transformers import PreTrainedTokenizerFast, LlamaForCausalLM
from diffusers import HiDreamImagePipeline
PreTrainedTokenizerFast.from_pretrained('meta-llama/Llama-3.1-8B-Instruct')
LlamaForCausalLM.from_pretrained('meta-llama/Llama-3.1-8B-Instruct',
    torch_dtype='auto')
HiDreamImagePipeline.from_pretrained('HiDream-ai/HiDream-I1-Dev')
"

COPY handler_hidream.py /app/handler.py
CMD ["python", "/app/handler.py"]
```

Container 2: Blender 5.0 + MPFB2 Worker

```dockerfile
```

```
FROM ubuntu:24.04

# Install system dependencies
RUN apt-get update && apt-get install -y \
    wget xz-utils libx11-6 libgl1 libglib2.0-0 python3 python3-pip \
    && rm -rf /var/lib/apt/lists/*

# Install Blender 5.0 headless
RUN wget -q https://download.blender.org/release/Blender5.0/blender-5.0-linux-x64.ta
    && tar xf blender-5.0-linux-x64.tar.xz -C /opt/ \
    && ln -s /opt/blender-5.0-linux-x64/blender /usr/local/bin/blender \
    && rm blender-5.0-linux-x64.tar.xz

# Install Python packages into Blender's Python
RUN /opt/blender-5.0-linux-x64/5.0/python/bin/python3 -m pip install \
    runpod boto3 numpy

# Install MPFB2 extension
COPY mpfb2_extension/ /opt/blender-5.0-linux-x64/5.0/extensions/mpfb/

# Copy all CC0 asset packs (~2-3 GB)
COPY mpfb_assets/ /app/mpfb_assets/

# Copy pipeline scripts
COPY blender_scripts/ /app/blender_scripts/
COPY handler_blender.py /app/handler.py

CMD ["python3", "/app/handler.py"]
```

Appendix B: Spoke App File Structure

```
spoke_app_skeleton/    (forked from kokokino/spoke_app_skeleton)
├── .meteor/
├── client/
|   ├── main.js
|   └── views/
|       ├── AvatarGenerator.js      # Prompt input + image upload
|       ├── AvatarPreview.js        # Babylon.js 3D viewer
|       ├── AvatarGallery.js        # User's saved avatars
|       ├── AvatarCustomizer.js     # Body/face sliders (future)
```

```
│       └── GenerationProgress.js   # Real-time progress bar
├── imports/
│   ├── api/
│   │   ├── avatars/
│   │   │   ├── collection.js       # AvatarsCollection
│   │   │   ├── methods.js          # generate, download, delete
│   │   │   └── publications.js     # user's avatars
│   │   └── jobs/
│   │       ├── collection.js       # JobsCollection
│   │       ├── methods.js          # createJob, cancelJob
│   │       └── publications.js     # reactive job status
│   └── lib/
│       ├── runpod-client.js        # RunPod API integration
│       ├── b2-client.js            # Backblaze B2 upload/URL generation
│       ├── babylon-loader.js       # Babylon.js GLB + morph targets
│       └── prompt-decomposer.js    # LLM prompt → MPFB2 params
├── server/
│   ├── main.js
│   ├── avatar-pipeline.js          # Orchestrate generation steps
│   └── runpod-webhooks.js          # Receive job completion callbacks
├── public/
│   └── sample-avatars/             # Pre-generated demo GLBs
├── CLAUDE.md                       # Claude Code guidance
├── package.json
├── settings.json                   # RunPod keys, B2 keys, model configs
└── rspack.config.js
```